Kazakh-British Technical University

Data collection and preparation

STUDENTS' INDEPENDENT STUDY (SIS) №1

Do Wealthier Countries Win More Olympic Medals?

Full Name: Shauyenov Yerden

ID: 24B031128

05.10.2026

Almaty

Task 1. Data collection: API

I used the open World Bank API (no API key, JSON format): https://api.worldbank.org/v2/country/{iso3}/indicator/{indicator}. Two indicators are requested for every country: NY.GDP.MKTP.CD (GDP, current US$) and SP.POP.TOTL (total population).

Step 1: define the 12 countries with their ISO-3 codes (United States, China, Germany, Japan, United Kingdom, France, Russian Federation, Australia, Kazakhstan, Norway, Kenya, Jamaica) - chosen to span a wide range of GDP, population and medal counts.

```
countries = [
    {"country": "United States",      "iso3": "USA"},
    {"country": "China",              "iso3": "CHN"},
    {"country": "Germany",            "iso3": "DEU"},
    {"country": "Japan",              "iso3": "JPN"},
    {"country": "United Kingdom",     "iso3": "GBR"},
    {"country": "France",             "iso3": "FRA"},
    {"country": "Russian Federation", "iso3": "RUS"},
    {"country": "Australia",          "iso3": "AUS"},
    {"country": "Kazakhstan",         "iso3": "KAZ"},
    {"country": "Norway",             "iso3": "NOR"},
    {"country": "Kenya",              "iso3": "KEN"},
    {"country": "Jamaica",            "iso3": "JAM"},
]
len(countries)
```

```
12
```

Step 2: send a request for one country and examine the response. The function fetch_indicator() calls requests.get() with the parameters format=json and mrnev=1 (the most recent non-empty value), checks the status with raise_for_status() and takes the record from the JSON list (payload[0] is metadata, payload[1] holds the data). Test request for the United States: GDP 30,769,700,000,000 US$ and population 341,784,857, both for 2025.

```python
def fetch_indicator(iso3, indicator):
    url = f"https://api.worldbank.org/v2/country/{iso3}/indicator/{indicator}"
    params = {"format": "json", "mrnev": 1}
    response = requests.get(url, params=params, timeout=30)
    response.raise_for_status()
    payload = response.json()

    records = payload[1] #payload[0] is metadata
    record = records[0]
    return record["date"], record["value"]

def fetch_economy(country, iso3):
    gdp_year, gdp = fetch_indicator(iso3, "NY.GDP.MKTP.CD")
    pop_year, pop = fetch_indicator(iso3, "SP.POP.TOTL")
    return {
        "country": country,
        "iso3": iso3,
        "gdp_current_usd": gdp,
        "gdp_year": gdp_year,
        "population": pop,
        "population_year": pop_year,
    }

#quick test
fetch_economy("United States", "USA")
```

[28]

```
{'country': 'United States',
 'iso3': 'USA',
 'gdp_current_usd': 30769700000000,
 'gdp_year': '2025',
 'population': 341784857,
 'population_year': '2025'}
```

Step 3: collect the data for all 12 countries in a loop (1-second pause between requests, try/except for failed requests). All 12 requests succeeded. The result economy_raw has 12 rows and 6 columns (country, iso3, gdp_current_usd, gdp_year, population, population_year); all values are for 2025. Saved as economy_raw.csv.

country	iso3	gdp_current_usd	population
United States	USA	3.076970e+13	341784857
China	CHN	1.949804e+13	1406585000
Germany	DEU	5.050923e+12	83491249
Japan	JPN	4.435163e+12	123366734
United Kingdom	GBR	4.002588e+12	69487000
France	FRA	3.366316e+12	68720337
Russian Federation	RUS	2.561310e+12	143513328
Australia	AUS	1.798519e+12	27614411
Kazakhstan	KAZ	3.062392e+11	20843754
Norway	NOR	5.307557e+11	5610870
Kenya	KEN	1.359413e+11	57532493
Jamaica	JAM	2.270490e+10	2837077

```python
economy_records = []

for c in countries:
    try:
        record = fetch_economy(c["country"], c["iso3"])
        economy_records.append(record)
        print(f"OK: {c['country']}")
    except Exception as e:
        print(f"FAIL {c['country']: {e}}")
    time.sleep(1)

economy_raw = pd.DataFrame(economy_records)
economy_raw.to_csv("economy_raw.csv", index=False)
economy_raw
```

[29]

```
OK: United States
OK: China
OK: Germany
OK: Japan
OK: United Kingdom
OK: France
OK: Russian Federation
OK: Australia
OK: Kazakhstan
OK: Norway
OK: Kenya
OK: Jamaica
```

Task 2. Data collection: web scraping

Static webpage: Wikipedia, “All-time Olympic Games medal table”: https://en.wikipedia.org/wiki/All-time_Olympic_Games_medal_table - one large HTML table with one row per country (NOC) and the medal counts of the Summer Games, the Winter Games and in total.

Step 1: request the page. Only this one public page is requested (no crawling), with requests.get(), a timeout and a descriptive User-Agent header with a contact address, in line with Wikipedia's robots.txt; raise_for_status() checks the response and BeautifulSoup parses the HTML.

Step 2: find the right table. The page contains 36 tables with the class “wikitable”; the full medal table is the largest of them (165 rows), so it is selected with max(tables, key=lambda t: len(t.find_all("tr"))).

Step 3: extract the fields from every row. The first cell is the country name; footnote markers such as “United States[P][Q][R][Z][F]” are removed (sup tags and a regular expression). The numeric cells are grouped as Summer, Winter and Combined, five values each (No. of Games, Gold, Silver, Bronze, Total), so the Summer total is the 5th number of a row (e.g. Afghanistan: 16, 0, 0, 2, 2 → total 2). Result: 163 country rows (all_medal_rows).

Step 4: select the 12 countries. The Olympic table uses NOC names, so lookup_medals() also tries an alias dictionary (NOC_ALIASES): United Kingdom → Great Britain, Russian Federation → Russia. Result olympics_raw: 12 rows and 2 columns (country,

summer_medals_alltime), saved as olympics_raw.csv.

country	summer_medals_alltime
United States	2774
China	727
Germany	687
Japan	542
United Kingdom	986
France	824
Russian Federation	421
Australia	594
Kazakhstan	78
Norway	169
Kenya	124
Jamaica	94

```python
def lookup_medals(country, all_rows):
    candidates = [country, NOC_ALIASES.get(country)]
    for row in all_rows:
        if row["noc_name"] in candidates:
            return row["summer_medals_alltime"]
    return None

olympics_records = [
    {"country": c["country"], "summer_medals_alltime": lookup_medals(c["country"], all_medal_rows)}
    for c in countries
]

olympics_raw = pd.DataFrame(olympics_records)
olympics_raw.to_csv("olympics_raw.csv", index=False)
olympics_raw
```

[31]

	country	summer_medals_alltime
0	United States	2774
1	China	727
2	Germany	687
3	Japan	542
4	United Kingdom	986
5	France	824
6	Russian Federation	421
7	Australia	594
8	Kazakhstan	78

Task 3. Data cleaning

Step 1: check data types, missing values and duplicates in both datasets.

Dataset	Rows	Missing values	Duplicate countries	Data types
economy_raw	12	0	0	float64: gdp_current_usd; int64: population; text: country, iso3, gdp_year, population_year
olympics_raw	12	0	0	int64: summer_medals_alltime; text: country

```
for name, frame in [("economy_raw", economy_raw), ("olympics_raw", olympics_raw)]:
    print(f"--- {name} ---")
    print(frame.dtypes)
    print("Missing values:\n", frame.isna().sum())
    print("Duplicate countries:", frame["country"].duplicated().sum())
    print()
```

[32]

```
--- economy_raw ---
country               str
iso3                  str
gdp_current_usd     float64
gdp_year              str
population          int64
population_year       str
dtype: object
Missing values:
 country           0
iso3               0
gdp_current_usd    0
gdp_year           0
population         0
population_year    0
dtype: int64
Duplicate countries: 0

--- olympics_raw ---
country                   str
summer_medals_alltime   int64
dtype: object
```

Step 2: make sure the numeric columns are numeric (pd.to_numeric with errors="coerce") and drop rows with missing values. Rows before / after dropping: (12, 12) → (12, 12), so no records were lost.

```
economy = economy_raw.copy()
olympics = olympics_raw.copy()

economy["gdp_current_usd"] = pd.to_numeric(economy["gdp_current_usd"], errors="coerce")
economy["population"] = pd.to_numeric(economy["population"], errors="coerce")
olympics["summer_medals_alltime"] = pd.to_numeric(olympics["summer_medals_alltime"], errors="coerce")

before = len(economy), len(olympics)
economy = economy.dropna(subset=["gdp_current_usd", "population"])
olympics = olympics.dropna(subset=["summer_medals_alltime"])
print("Rows before/after dropping missing values: ", before, "->", (len(economy), len(olympics)))
```

[33]

```
Rows before/after dropping missing values:  (12, 12) -> (12, 12)
```

Task 4. Merging

Step 1: combine the two datasets with an inner join on the column country (the names were already harmonized in Task 2). Then rename summer_medals_alltime → medals and add two helper columns: gdp_trillions = gdp_current_usd / 1e12 and population_millions = population / 1e6.

Result: 12 rows × 9 columns - all 12 countries matched. Saved as merged_economy_olympics.csv.

country	gdp_trillions	population_millions	medals
United States	30.77	341.8	2774
China	19.50	1406.6	727

country	gdp_trillions	population_millions	medals
Germany	5.05	83.5	687
Japan	4.44	123.4	542
United Kingdom	4.00	69.5	986
France	3.37	68.7	824
Russian Federation	2.56	143.5	421
Australia	1.80	27.6	594
Kazakhstan	0.31	20.8	78
Norway	0.53	5.6	169
Kenya	0.14	57.5	124
Jamaica	0.02	2.8	94

```
[34]
df = pd.merge(economy, olympics, on="country", how="inner")
df = df.rename(columns= {"summer_medals_alltime": "medals"})
df["gdp_trillions"] = df["gdp_current_usd"] / 1e12
df["population_millions"] = df["population"] / 1e6

df.to_csv("merged_economy_olympics.csv", index=False)
print(df.shape)
df[["country", "gdp_trillions", "population_millions", "medals"]]
```

```
(12, 9)
```

	country	gdp_trillions	population_millions	medals
0	United States	30.769700	341.784857	2774
1	China	19.498039	1406.585000	727
2	Germany	5.050923	83.491249	687
3	Japan	4.435163	123.366734	542
4	United Kingdom	4.002588	69.487000	986
5	France	3.366316	68.720337	824
6	Russian Federation	2.561310	143.513328	421
7	Australia	1.798519	27.614411	594
8	Kazakhstan	0.306239	20.843754	78
9	Norway	0.530756	5.610870	169
10	Kenya	0.135941	57.532493	124
11	Jamaica	0.022705	2.837077	94

Task 5. Analysis

Step 1: descriptive statistics of gdp_trillions, population_millions and medals (describe()).

	gdp_trillions	population_millions	medals
count	12	12	12
mean	6.04	195.9	668.33
std	9.40	392.1	730.47
min	0.02	2.8	78

	gdp_trillions	population_millions	medals
25%	0.47	25.9	157.75
50%	2.96	69.1	568
75%	4.59	128.4	751.25
max	30.77	1406.6	2774

The mean is far above the median for GDP (6.04 vs. 2.96) and for population (195.9 vs. 69.1), because a few very large countries (the United States, China) dominate - both variables are strongly right-skewed.

```
[35]
1  df[["gdp_trillions", "population_millions", "medals"]].describe()
```

	gdp_trillions	population_millions	medals
count	12.000000	12.000000	12.000000
mean	6.039850	195.948926	668.333333
std	9.399784	392.130562	730.467513
min	0.022705	2.837077	78.000000
25%	0.474627	25.921747	157.750000
50%	2.963813	69.103668	568.000000
75%	4.589103	128.403382	751.250000
max	30.769700	1406.585000	2774.000000

Step 2: Pearson correlation (r) with p-values between each predictor and the number of medals, on raw values and on log10-transformed values (the log transform is used because GDP and population span several orders of magnitude).

Pair	Pearson r	p-value
GDP vs. medals (raw)	0.873	< 0.001
Population vs. medals (raw)	0.233	0.465
log GDP vs. log medals	0.905	< 0.001
log population vs. log medals	0.694	0.012

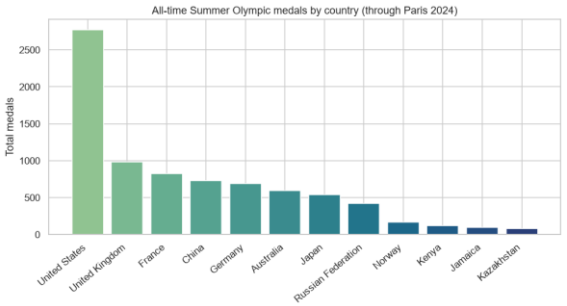
```
[36]
1  df["log_gdp"] = np.log10(df["gdp_current_usd"])
2  df["log_population"] = np.log10(df["population"])
3  df["log_medals"] = np.log10(df["medals"])
4
5  r_gdp, p_gdp = stats.pearsonr(df["gdp_current_usd"], df["medals"])
6  r_pop, p_pop = stats.pearsonr(df["population"], df["medals"])
7  r_lgdp, p_lgdp = stats.pearsonr(df["log_gdp"], df["log_medals"])
8  r_lpop, p_lpop = stats.pearsonr(df["log_population"], df["log_medals"])
9
10 print(f"Raw GDP vs medals:            r = {r_gdp:.3f}, p = {p_gdp:.3f}")
11 print(f"Raw Population vs medals:     r = {r_pop:.3f}, p = {p_pop:.3f}")
12 print(f"Log GDP vs Log medals:        r = {r_lgdp:.3f}, p = {p_lgdp:.3f}")
13 print(f"Log Population vs Log Medals: r = {r_lpop:.3f}, p = {p_lpop:.3f}")
```

```
Raw GDP vs medals:            r = 0.873, p = 0.000
Raw Population vs medals:     r = 0.233, p = 0.465
Log GDP vs Log medals:        r = 0.905, p = 0.000
Log Population vs Log Medals: r = 0.694, p = 0.012
```

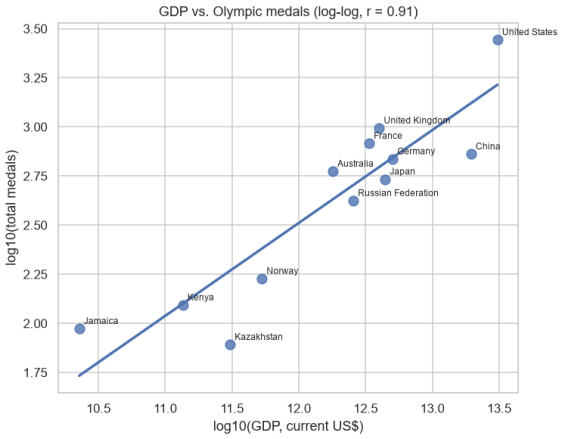
Task 6. Visualization

Four plots were built with Matplotlib and Seaborn, each with a title and labeled axes:

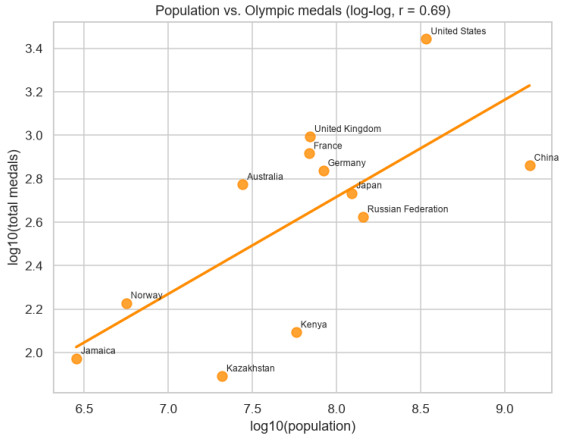
- Plot 1 - bar chart: all-time Summer Olympic medals by country.
- Plot 2 - scatter plot with a trend line: GDP vs. medals (log-log scale, r = 0.91).
- Plot 3 - scatter plot with a trend line: population vs. medals (log-log scale, r = 0.69).
- Plot 4 - bar chart "Olympic efficiency": medals per $100B of GDP.



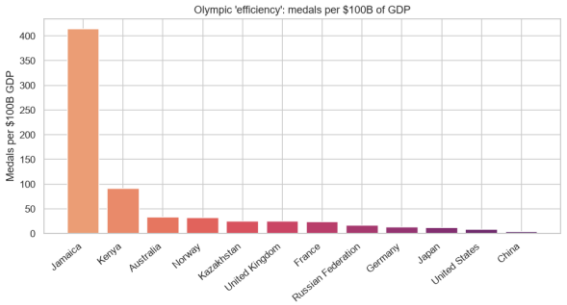
Plot 1. All-time Summer Olympic medals by country.



Plot 2. GDP vs. Olympic medals (log-log).



Plot 3. Population vs. Olympic medals (log-log).



Plot 4. Medals per $100B of GDP.

Task 7. Findings and conclusions

1. **GDP is a strong predictor of Olympic success.** r = 0.87 on raw values and r = 0.91 on the log-log scale, both with $p < 0.001$.
2. **Population alone is a much weaker predictor.** r = 0.23 on raw values (p = 0.47, not significant); on the log-log scale r = 0.69 (p = 0.012), still below GDP. China has the largest population (1,406.6 million) but only 727 medals, versus 2,774 for the United States (341.8 million).
3. **Some countries strongly over- or underperform their economic size.** Jamaica (about 414 medals per $100B of GDP) and Kenya (about 91) win far more than their GDP suggests, compared with about 9 for the United States and about 4 for China, the lowest value.
4. **The data are strongly skewed.** GDP ranges from 0.02 to 30.77 trillion US$ and the mean is about twice the median, so log transformation was needed and the raw correlations are driven mainly by the United States and China.
5. **Overall,** economic resources are the strongest single predictor tested here, but sport-specific talent pipelines and historical investment clearly matter too.

Limitations and future work:

- Country names were harmonized with a small manual mapping (2 entries) plus footnote removal; a larger country list would need a fuller alias table.
- Wikipedia lists predecessor teams (e.g. the Soviet Union, East and West Germany, unified teams) separately and does not combine them with successor countries, so the totals of Russia (421) and Germany (687) cover the current national team only.
- The 12 countries are a convenience sample chosen to span a wide range, not a random sample; a larger set would make the estimates more reliable.

Data sources: World Bank Open Data, https://data.worldbank.org/ (indicators NY.GDP.MKTP.CD and SP.POP.TOTL);
Wikipedia, “All-time Olympic Games medal table”, https://en.wikipedia.org/wiki/All-time_Olympic_Games_medal_table, accessed 2026.

GitHub: https://github.com/yerden-shauyenov/DCP-MidTerm